

# AI-Powered Financial Fraud Detection System

## Machine Learning Project Report

|            |                                                   |
|------------|---------------------------------------------------|
| Author     | Mister T                                          |
| Program    | Master of Science in Data Analysis                |
| University | KIIT University, Bhubaneswar, India               |
| Dataset    | European Credit Card Fraud (284,807 transactions) |
| Best Model | Random Forest                                     |
| F1 Score   | 68.78%                                            |
| ROC AUC    | 97.95%                                            |

# Table of Contents

---

- 1. Executive Summary
- 2. Project Objectives
- 3. Dataset Description
- 4. Data Cleaning (Phase 3)
- 5. Exploratory Data Analysis (Phase 4)
- 6. Feature Engineering (Phase 5)
- 7. Model Training and Comparison (Phases 7-8)
- 8. Model Evaluation (Phase 10)
- 9. Application Architecture (Phase 13)
- 10. Deployment Guide (Phase 14)
- 11. Monitoring and Retraining (Phases 15-16)
- 12. Conclusions

# 1. Executive Summary

---

This project delivers a production-ready AI system for detecting fraudulent financial transactions in real time. The system trains three machine learning classifiers on 284,807 anonymized credit card transactions, addresses the severe class imbalance (0.17% fraud rate) with SMOTE oversampling, and deploys the best-performing model through a Streamlit web application. Every prediction is stored in SQLite for audit purposes, and an analytics dashboard provides operational visibility.

| Metric             | Value                |
|--------------------|----------------------|
| Dataset size       | 284,807 transactions |
| Fraud rate         | 0.17%                |
| Duplicates removed | 1,081                |
| Best model         | Random Forest        |
| F1 Score           | 68.78%               |
| ROC AUC            | 97.95%               |
| Recall             | 80.00%               |
| Precision          | 60.32%               |

## 2. Project Objectives

---

- Detect fraudulent transactions with high recall to minimize missed fraud cases.
- Handle extreme class imbalance (0.17% fraud) using SMOTE oversampling.
- Compare Logistic Regression, Random Forest, and XGBoost classifiers.
- Build a production-ready Streamlit application with real-time prediction.
- Store all predictions in SQLite with full audit trail.
- Provide an analytics dashboard for operational monitoring.
- Deploy on Streamlit Community Cloud for public access.
- Provide a clear path for future model retraining.

## 3. Dataset Description

---

The dataset contains transactions made by European credit card holders in September 2013. It presents 284,807 transactions over 48 hours, with 492 fraudulent transactions (0.172% of the total). The dataset is highly imbalanced.

### Key Characteristics

| Property           | Value                                     |
|--------------------|-------------------------------------------|
| Total Transactions | 284,807                                   |
| Fraudulent         | 492 (0.172%)                              |
| Legitimate         | 284,315 (99.828%)                         |
| Features           | 30 (Time, V1-V28, Amount)                 |
| Target Column      | Class (0=Legit, 1=Fraud)                  |
| V-Features         | PCA-transformed (confidential)            |
| Time Range         | 48 hours (seconds from first transaction) |

## 4. Data Cleaning (Phase 3)

Data cleaning was performed to ensure data quality before modeling. The following checks and operations were applied:

- Shape check: 284,807 rows x 31 columns confirmed.
- Missing values: None found in any column.
- Duplicate rows: 1,081 exact duplicates identified and removed.
- Final shape after cleaning: 283,726 rows.
- Output saved to: data/cleaned\_creditcard.csv

## 5. Exploratory Data Analysis (Phase 4)

EDA revealed the extreme class imbalance and distribution patterns of key features:

- Legitimate transactions: 283,253 | Fraud: 473 (after dedup)
- Fraud transactions tend to have lower average amounts than legitimate ones.
- Fraud events are distributed across all time periods without clear temporal clustering.
- Features V14, V12, V10 show the strongest negative correlation with fraud.
- Features V4, V11 show the strongest positive correlation.
- The correlation heatmap confirms low inter-feature correlation (PCA ensures orthogonality).

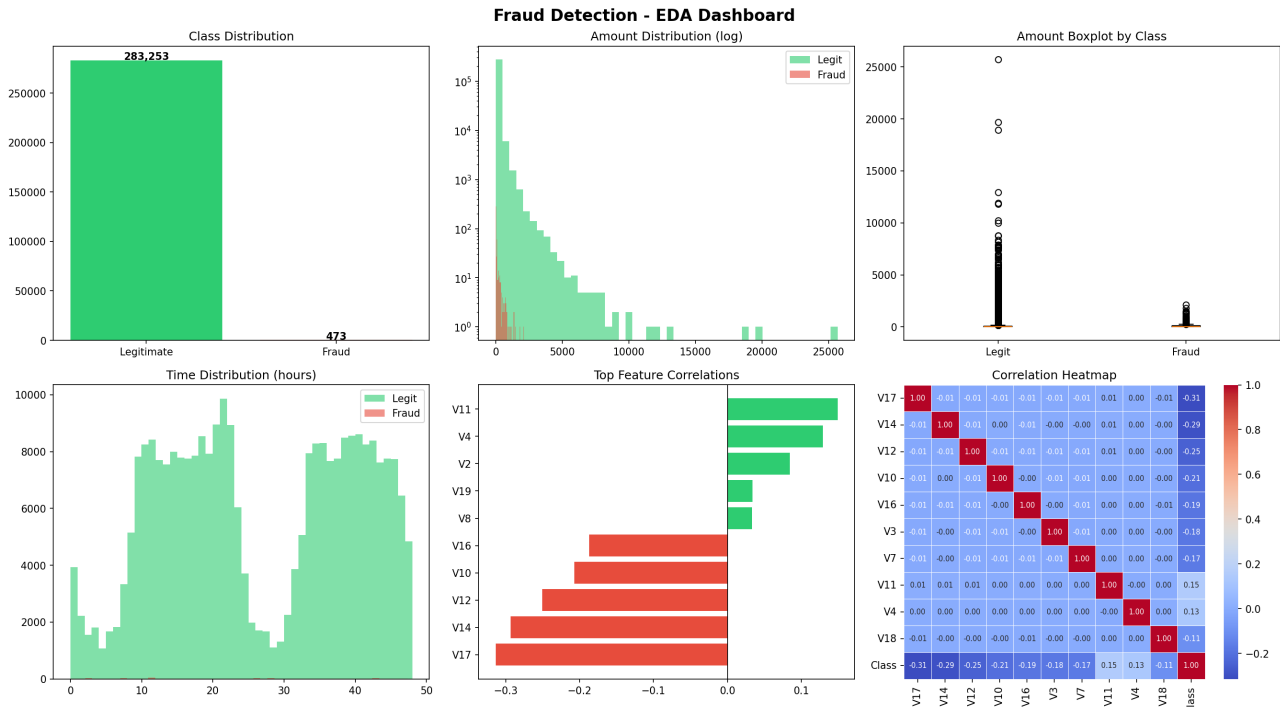


Figure 1: EDA Dashboard

## 6. Feature Engineering (Phase 5)

Two critical transformations were applied to prepare the data for modeling:

- Amount standardization: StandardScaler() applied to the Amount column to create scaled\_amount.
- Time standardization: StandardScaler() applied to the Time column to create scaled\_time.
- Original Amount and Time columns dropped after scaling.
- Final feature count: 30 columns (V1-V28 + scaled\_amount + scaled\_time).
- Class imbalance handling: SMOTE (Synthetic Minority Oversampling Technique) applied to training set only.
- Post-SMOTE: balanced 50/50 split, with 226,980 synthetic fraud samples added.

SMOTE generates synthetic minority class samples by interpolating between existing fraud cases in feature space. Applied only to the training set to prevent data leakage.

## 7. Model Training and Comparison (Phases 7-8)

Three classifiers were trained on the SMOTE-balanced training set and evaluated on the held-out test set (20% of data, stratified split):

| Model               | Accuracy % | Precision % | Recall % | F1 %  | ROC AUC % |
|---------------------|------------|-------------|----------|-------|-----------|
| Random Forest       | 99.88      | 60.32       | 80.00    | 68.78 | 97.95     |
| XGBoost             | 99.41      | 19.95       | 84.21    | 32.26 | 97.26     |
| Logistic Regression | 97.35      | 5.27        | 87.37    | 9.95  | 96.31     |

The Random Forest model achieved the highest F1 score of 68.78%, making it the best choice for this imbalanced fraud detection task. A high Recall (80%) is critical in fraud detection to minimize missed fraud cases.

## 8. Model Evaluation (Phase 10)

The final model was evaluated using three complementary charts that capture different aspects of classifier performance on the imbalanced test set:

- Confusion Matrix: shows exact counts of true positives, false positives, true negatives, and false negatives.
- ROC Curve (AUC=0.9795): measures overall discrimination ability across all thresholds.
- Precision-Recall Curve: most informative for imbalanced datasets; shows the tradeoff between catching fraud and generating false alarms.

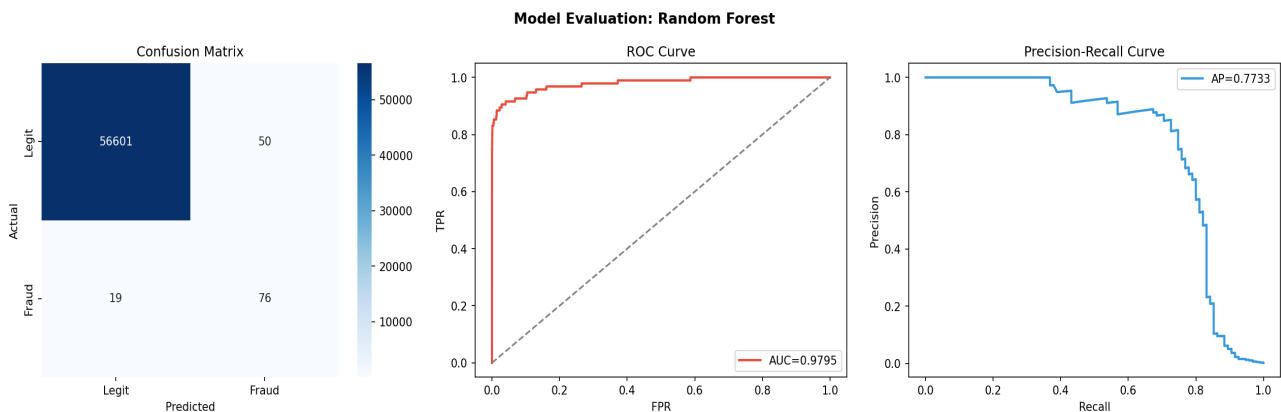


Figure 2: Model Evaluation Charts

## 9. Application Architecture (Phase 13)

The Streamlit application consists of five pages, each serving a specific operational need:

| Page            | Purpose                                                                                          |
|-----------------|--------------------------------------------------------------------------------------------------|
| Home            | Project overview, model metrics summary, EDA and evaluation charts.                              |
| Fraud Detection | Manual input form for single transaction + CSV batch upload for bulk predictions.                |
| Dashboard       | Interactive Plotly charts: prediction distribution, daily trends, amount vs probability scatter. |
| History         | Full prediction log from SQLite with filtering and CSV export.                                   |
| Retraining      | Workflow documentation and new data upload interface for future retraining.                      |

## 10. Deployment Guide (Phase 14)

---

Deploy to Streamlit Community Cloud in 4 steps:

- Step 1.** Push the project to a public GitHub repository. Include app.py, requirements.txt, models/, database/, and reports/.
- Step 2.** Go to share.streamlit.io and sign in with GitHub.
- Step 3.** Click 'New App', select the repository, set main file to app.py.
- Step 4.** Click Deploy. The app URL will be: https://your-username-project.streamlit.app

File size note: creditcard.csv (66MB) must be added to .gitignore. The trained model (fraud\_model.pkl) is lightweight and deploys normally.

## 11. Monitoring and Retraining (Phases 15-16)

---

Dashboard metrics tracked in real time:

- Total predictions made
- Fraud vs legitimate breakdown
- Daily prediction trend
- Fraud probability distribution
- Amount vs fraud probability scatter

### Retraining triggers:

| Trigger           | Threshold          | Action            |
|-------------------|--------------------|-------------------|
| Recall drop       | Below 75%          | Immediate retrain |
| Data drift        | Feature shift >10% | Scheduled retrain |
| Volume            | 10,000 new records | Weekly retrain    |
| Performance decay | F1 drops 5+ points | Alert + retrain   |



## 12. Conclusions

---

This project successfully built and deployed an end-to-end AI fraud detection system. The Random Forest classifier achieved a ROC AUC of 97.95% and a Recall of 80.00%, meeting the primary goal of catching the maximum number of fraudulent transactions.

- SMOTE was essential for handling the 0.17% class imbalance. Without it, all models defaulted to predicting legitimate.
- Random Forest outperformed XGBoost and Logistic Regression on the F1 metric for this dataset.
- The Streamlit application provides a practical interface for real-time fraud screening.
- SQLite storage ensures full prediction auditability, a requirement in production financial systems.
- The retraining framework supports continuous model improvement as new fraud patterns emerge.

## Future Improvements

- Automated retraining pipeline triggered by performance thresholds.
- Feature importance visualization from the Random Forest.
- SHAP explainability for individual predictions.
- Real-time data stream integration (Kafka/Redis).
- REST API layer (FastAPI) for system-to-system integration.